

MAIE 6000C: Engineering AI Products: From Prototype to Production

Week 5 — Lab

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lfteris NTAFL0S, PhD

The Hong Kong University of Science and Technology



Today's Rule: Observe the Contract Before You Extend It

An API is not just a URL.

Endpoint = path + method + request + response + validation + errors + side effects + security + retry behavior

Question for today

- What can a caller send, receive, rely on, and recover from?

Session 1 Plan and Team Roles

Session 1 Plan

- Observe the running API
- Inspect Swagger / OpenAPI documentation
- Create a valid case
- Trigger a validation error
- Retrieve existing and missing records
- Observe worker-driven state changes
- Trace route → schema → service → test
- Build evidence maps as a team

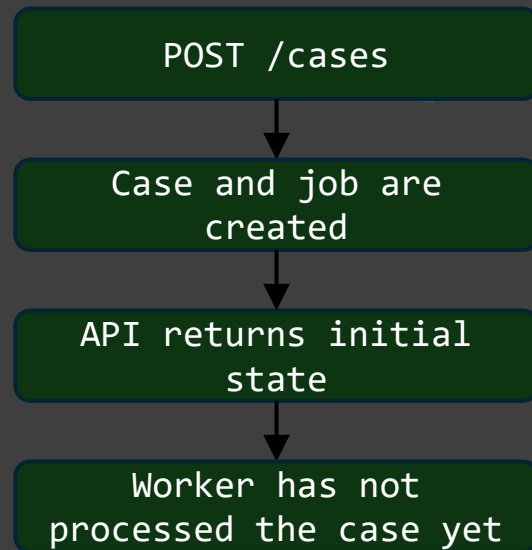
Activity: Assign team roles (2 min)

- **Driver:** run commands
- **Evidence recorder(s):** capture paths, functions, responses
- **Skeptic/tester(s):** ask “How do we know?” and check assumptions

Start the Starter System

Keep the worker stopped at first.

Why?



```
# 1. Open docker desktop
# 2. Open Powershell / Bash Terminal
# 3. cd into working folder
docker compose up --build -d
docker compose ps
docker compose stop worker

# can also keep API Swagger page open:
http://localhost:8000/docs
```

Live Demo Step 1: Health, Swagger, and OpenAPI

Check Service Health and API Documentation

PowerShell

```
curl.exe -i http://localhost:8000/health/live  
curl.exe -i http://localhost:8000/health/ready
```

Bash

```
curl -i http://localhost:8000/health/live  
curl -i http://localhost:8000/health/ready
```

Open in a browser

```
http://localhost:8000/docs  
http://localhost:8000/openapi.json
```

Observe

- Which routes are exposed?
- Which request fields are required?
- Which response models are shown?
- Which status codes are documented?

Live Demo Step 2: Create a Case Through the AP

Create a Valid Case

PowerShell

```
Invoke-RestMethod -Uri "http://localhost:8000/cases" -Method Post -ContentType "application/json" -Body '{"title":"Payment Processing Delay","description":"Customers are experiencing unusually long delays when completing online payments."}' | ConvertTo-Json -Depth 10
```

Bash

```
curl -s -X POST "http://localhost:8000/cases" -H "Content-Type: application/json" -d '{"title":"Payment Processing Delay","description":"Customers are experiencing unusually long delays when completing online payments."}' | python -m json.tool
```

Record

- HTTP status code
- Case ID
- Job ID, if returned
- Initial case status
- Fields visible to the client

Live Demo Step 3: Trigger a Validation Error

Send Valid JSON With Missing Required Data

```
# PowerShell
$invalidBody = '{}'  
  
curl.exe -i -sS -X POST  
"http://localhost:8000/cases" \  
-H "Content-Type: application/json" \  
--data $invalidBody
```

```
# Bash  
  
curl -i -sS -X POST  
"http://localhost:8000/cases" \  
-H "Content-Type: application/json" \  
--data '{}'
```

Observe

- Does the API return a 422 response?
- Is the error body structured?
- Does the response identify the invalid or missing fields?
- Does the API expose internal implementation details?

Live Demo Step 4: Retrieve Current State and a Missing Record

Retrieve a Known Case

```
# PowerShell  
curl.exe -i http://localhost:8000/cases/CASE_ID
```

```
# Bash  
curl -i http://localhost:8000/cases/CASE_ID
```

Retrieve a Missing Case

```
# PowerShell  
curl.exe -i http://localhost:8000/cases/MISSING_CASE_ID
```

```
# Bash  
curl -i http://localhost:8000/cases/MISSING_CASE_ID
```

Live Demo Step 5: Start the Worker and Observe the API Boundary

Start Background Processing

Terminal 1

```
# PowerShell / Bash
docker compose start worker
docker compose logs -f worker # -f (follow: keeps the
terminal open and attached)
```

Retrieve the case again

```
# PowerShell (Bash: curl)
curl.exe -i http://localhost:8000/cases/CASE_ID
```

Terminal 2

```
# PowerShell / Bash
docker compose logs --tail=20 ai
```

Observe

```
Client → API → database/job
      ↓
      Worker → AI service
      ↓
      persisted result
      ↓
Client → API → updated state/result
```

Trace the Contract Through the Repository

Trace One Endpoint as a Connected System POST /cases

Request schema

services/common/schemas.py
CaseCreate
title: 3–200 characters
description: 5–4000 characters

API route

services/api/app/main.py
create_case(...)
POST /cases

Business function

services/api/app/main.py
create_case_with_job(...)

Database layer

services/common/models.py
Case + Job
services/common/db.py
get_db()

Response schemas

services/common/schemas.py
CaseCreateResponse
CaseRead + JobRead

Integration test

tests/integration/test_api_case_flow.py
test_create_and_read_case_and_job

Useful repository searches

```
git grep -n '"/cases"'
git grep -n 'CaseCreate'
git grep -n 'create_case_with_job'
git grep -n 'CaseCreateResponse'
git grep -n 'class Case'
git grep -n 'class Job'
git grep -n 'test_create_and_read_case_and_job'
git grep -n 'response_model'
git grep -n 'HTTPException'
```

Run the verification with:

```
docker compose exec api pytest -q tests/integration/test_api_case_flow.py
```

What Is the Contract?

What Did the Caller Learn?

Moment	What the caller should know
After POST	A record was created; current status; stable ID
While processing	Work is incomplete but visible
After completion	Result or final state is available
Invalid request	What must be corrected
Missing record	Resource does not exist
Processing failure	Failure is visible through current state

Key distinction

- Logs explain execution.
- The API communicates controlled system state.
- The database remains system truth over time.

Activity A: Build an API Contract Evidence Map

Team activity — 12 minutes

Open your repository and complete the evidence map.

Rules

- Do not write vague answers.
- Use actual paths, class names, function names, route decorators, and tests.
- Record observed behavior separately from intended behavior.
- If you cannot find evidence, write it as a gap or question.

Minimum evidence to find

- Workflow-starting API route
- Request validation schema
- Response model or response construction
- Function that creates the main record and job
- Missing-record behavior
- Validation-error behavior
- Integration test for the API flow
- One question or ambiguity

Activity B — API Behavior Test Matrix

Scenario	Minimum expectation
Valid create	Resource created; initial status returned
Retrieve created record	Current state returned
Missing required field	Validation response
Valid-format missing ID	Not-found response or observed alternative

Team activity — 8 minutes

- Run and record at least four cases.

Optional, only if route exists

- Retry invalid state
- Unauthorized request
- Invalid path parameter
- Worker-completed result retrieval

Record

- request sent;
- observed status code;
- response body shape;
- expected versus actual behavior;
- relevant code or test evidence.

Session 1

Checkpoint

Before Session 2, each team should have:

- ✓ API Contract Evidence Map started
- ✓ At least four observed API behaviors
- ✓ One route → schema → service → test trace
- ✓ One observed error response
- ✓ One API design question for the team project
- ✓ Notes saved in a shared working document

Keep these notes

Suggested locations:

- docs/architecture.md
- docs/api-contract.md



Session 2: From Evidence to Your System

From Observed API Behavior to Your Team's Interface Plan

Observed starter behavior



Project-specific endpoint contract



State and failure decisions



Tests for the Week 7 vertical slice

Goal

- Build a small interface plan that your team can explain, implement, test, and demonstrate.

Scope the Minimum Viable Interface

Start With Two Required Endpoints

1. POST /<main-resource>

- Start the primary workflow

2. GET /<main-resource>/{id}

- Retrieve current status and result

Add one endpoint only if essential

Examples:

- POST /<resource>/{id}/retry
- PATCH /<resource>/{id}
- POST /<resource>/{id}/review
- GET /<resource>?status=...

Scope rule

- Build one meaningful vertical slice before designing a platform.

Activity C: Draft the Endpoint Contracts

Team activity — 12 min

Define:

1. POST /<main-resource>
2. GET /<main-resource>/{id}
3. One optional action endpoint only if essential

For each endpoint, specify:

- caller and purpose;
- request fields;
- validation rules;
- success status and response;
- visible errors;
- records written or read;
- allowed state transition;
- authentication and retry notes;
- test that will verify it.

Use project nouns

- Good: /cases, /reports, /submissions, /reviews
- Avoid: /doEverything, /processData, /getAll

Activity D: Assign State Ownership, Errors, and Boundaries – Team Activity: 12 min

Who May Change What?

For your primary workflow, decide:

Question	Team decision
Which state transition does the API make?	
Which state transition belongs to the worker?	
What happens if input is invalid?	
What happens if the record does not exist?	
What happens if AI processing fails or times out?	
Which action must not be repeated accidentally?	
What is your authentication/authorization scope?	

Minimum scope decisions

- One client-visible failure path
- One retry or idempotency decision
- One realistic authorization statement
- One timeout/failure policy for the AI boundary

Activity E: Create a Week 7 Contract- Test Plan: Team activity — 12 minutes

Define at least three API tests.

Minimum tests

- 1. Success path**
Valid request creates the intended resource and initial state.
- 2. Validation failure**
Invalid or missing input returns a predictable client error.
- 3. Retrieval or state failure**
Missing resource, invalid transition, or failed processing is visible.

By Week 7, add an end-to-end test

POST resource

→ worker processes job

→ GET resource returns persisted result

For each test, record

- test name;
- setup;
- request;
- expected HTTP response;
- expected persisted state;
- owner;
- target date.

Peer Review: API Design Check – 5 min

Swap with another team

- Review against these questions:
- Can we identify one clear user workflow?
- Do the endpoint names match real entities?
- Is the create request different from the database model where necessary?
- Is the response truthful while processing is incomplete?
- Are at least one validation and one missing/failure path clear?
- Is ownership of state transitions explicit?
- Is the worker/AI boundary visible?
- Is the plan feasible for Week 7?

Give:

- **one strength**
- **one ambiguity**
- **one risk or recommendation**

Update the Real Repository Artifacts

Save the Work in the Repository

Minimum recommended updates

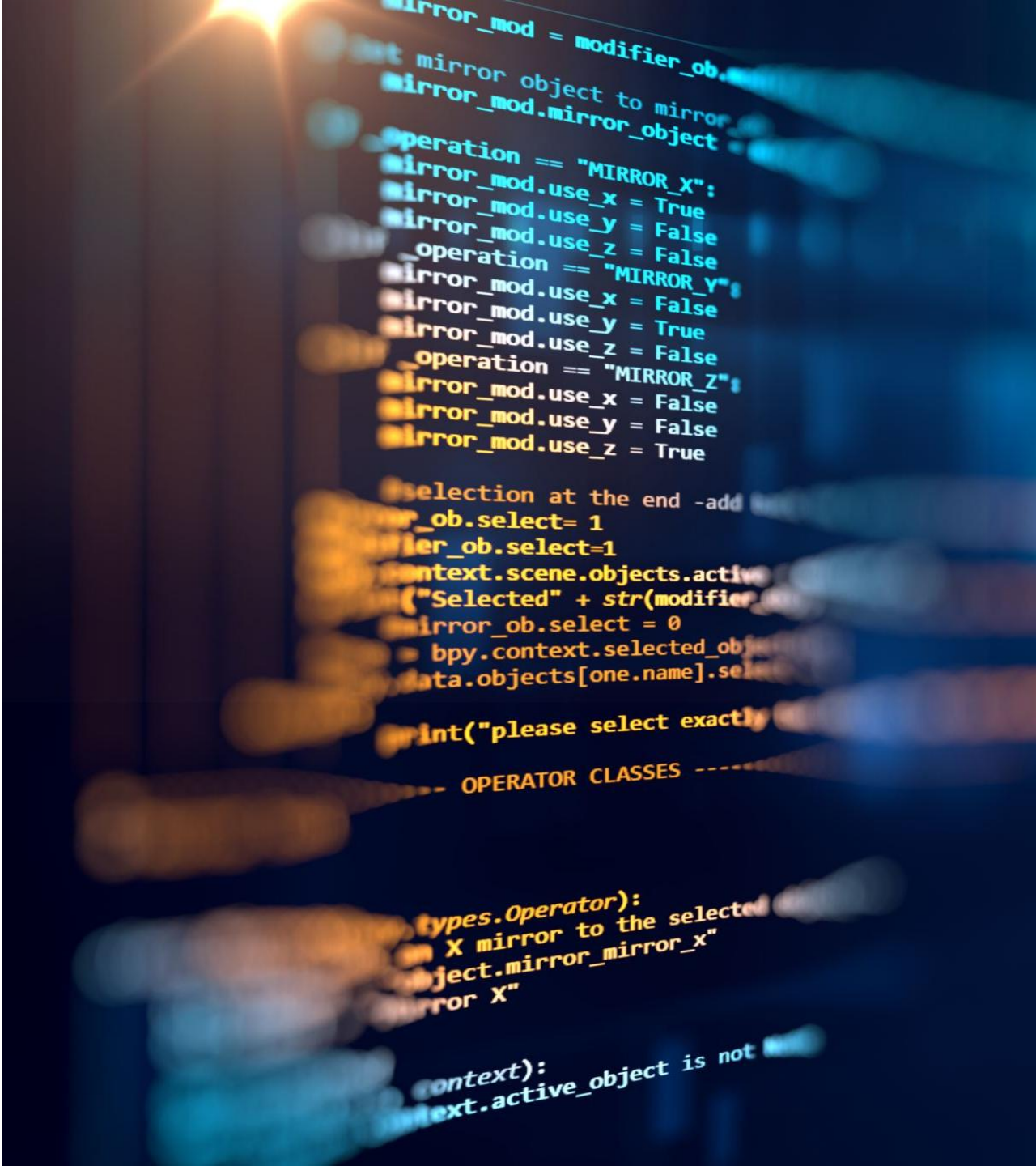
- docs/architecture.md

Add or refine:

- API/service interface section;
- primary endpoints;
- request/response examples;
- state ownership;
- error and failure behavior;
- Week 7 test plan.

Optional supporting artifact

- docs/api-contract.md



Exit Checklist and Next Week

Before You Leave

Each team should have:

- ✓ Evidence map based on actual starter code
- ✓ Observed API behavior matrix
- ✓ Two required endpoints defined
- ✓ One optional endpoint only if justified
- ✓ State-transition ownership table
- ✓ Error, timeout, retry, and auth-scope decisions
- ✓ At least three planned API tests
- ✓ Documentation owner for unfinished work
- ✓ Clear Week 7 vertical-slice target

Next Week: Background Jobs, Async Workflows, and Worker-Based Parallelism

Your API contract will become the boundary that the worker system must respect.